NLP & LOCAL LLM MINI-PROJECT

Bhasha2Code

A Multilingual AI Coding Assistant and Educational Sandbox

Bridging the Digital Linguistic Divide in Computer Science Education using Offline Natural Language Processing and Quantized Local LLMs.

STUDENT DETAILS

Name: Tejas Ingle
PRN: 72421415F
Department: Computer Engineering
Class & Year: T.E. Computer (2025–2026)

GUIDE & INSTITUTION

Project Guide: Prof. Gajanan Godage
College: Nutan Maharashtra Institute of Engg. & Tech.
Location: Talegaon Dabhade, Pune – 410507
Affiliation: Savitribai Phule Pune University (SPPU)

EXECUTIVE SUMMARY

Abstract & Project Overview

Project Abstract (Summary)

Bhasha2Code is an offline, privacy-preserving multilingual AI coding assistant engineered to eliminate the cognitive linguistic friction faced by non-native English engineering students.

By coupling a deterministic, offline Code-Mixed NLP Pipeline (Language, Intent, and Tone Classification) with a quantized local Large Language Model (Qwen2.5-Coder:1.5B via Ollama), the system accepts prompts in native and Romanized Marathi-English dialects.

The platform enforces strict AST-level syntax verification and executes verified programs in an isolated, resource-capped Docker sandbox. An integrated educational 'Study Mode' automatically generates algorithmic walkthroughs, ASCII flowcharts, and viva questions.

1. Cognitive Demarcation & Input NLP

Processes English, Marathi, Hindi, and transliterated code-mixed queries (e.g., 'Python madhye factorial kadha'). Translates unstructured linguistic intent into formal programming structures.

2. Offline Local Inference & Zero Cost

Operates entirely on consumer-grade CPU hardware via Ollama without cloud dependencies, API billing ($0 recurring cost), or privacy risks.

3. AST Verification & Safe Sandboxing

Prevents hallucinated or malformed syntax through automated Python AST checking, and executes verified code in an isolated Docker sandbox.

Problem Statement & Measurable Objectives

PROBLEM STATEMENT

Undergraduate engineering students who conceptualize computational logic in regional Indian languages (e.g., Marathi/Hindi) face severe cognitive friction when interacting with English-centric AI coding tools. Furthermore, commercial assistants mandate continuous high-speed internet, incur recurring subscription costs, raise proprietary code privacy concerns, and frequently output unverified syntax hallucinations without pedagogical explanations.

Objective 1: Multilingual Intent Understanding

Engineer an offline NLP pipeline using TF-IDF and calibrated linear classifiers to identify conversational languages and 18 programming intents across English, Marathi, Hindi, and Romanized code-mixing with >90% test accuracy.

Objective 2: Deterministic Prompt Synthesis

Formulate a policy-driven prompt engine that enforces the 'Code Preservation Mandate'—keeping programming syntax and keywords strictly in standard ASCII English while delivering explanations in the user's regional dialect.

Objective 3: Quantized Local LLM Inference

Deploy quantized, open-weights models (Qwen2.5-Coder:1.5B/3B) locally via Ollama to achieve sub-15s response latency on standard consumer-grade CPU hardware with zero cloud API billing.

Objective 4: AST Syntax Validation & Docker Sandbox

Implement an automated Abstract Syntax Tree (AST) validation pipeline to intercept syntax hallucinations, coupled with a resource-capped Docker container sandbox (512MB RAM, no network) for safe execution.

THEORETICAL FOUNDATION

Literature Survey & Related Work

Citation & Author	Paper Title & Venue	Proposed Methodology	Key Contribution	Identified Limitation
[1] M. Chen et al. (OpenAI, 2021)	Evaluating Large Language Models Trained on Code (Codex / HumanEval)	Fine-tuning GPT models on GitHub code repositories; introduces HumanEval benchmark.	Established functional correctness benchmark (pass@k) for AI code generation.	English-only prompt assumptions; requires massive cloud infrastructure; black-box API.
[2] B. Hui et al. (Alibaba, 2024)	Qwen2.5-Coder Technical Report: Advancing Code LLMs (arXiv:2409.12186)	Pre-training on 5.5T tokens with synthetic data and fill-in-the-middle tasks (1.5B to 32B).	State-of-the-art open-source coding performance in compact model sizes (1.5B/3B).	Lacks domain-specific prompt wrappers for Indian dialects; no built-in sandbox validation.
[3] T. Tiwary & P. Kumar (ACM, 2022)	Code-Mixing & Transliteration Challenges in Indian NLP (ACM TALIP)	Survey of linguistic tokenization and transliteration across Devanagari and Latin scripts.	Categorized matrix-language frame and phonetic ambiguities in Hindi/Marathi code-mixing.	Focused on social text and sentiment analysis; did not address programming intent synthesis.
[4] F. Pedregosa et al. (JMLR, 2011)	Scikit-learn: Machine Learning in Python (JMLR Vol. 12)	Efficient implementations of TF-IDF feature extraction, LinearSVC, and Logistic Regression.	Provided robust, sub-millisecond offline classification suitable for embedded execution.	Requires manual feature engineering and curated domain lexicons for code-mixed text.

Summary: While commercial models [1, 2] excel at English-to-code synthesis, they lack offline localized intent classifiers [3, 4] tailored for bilingual engineering students.

SYSTEM DIFFERENTIATION

Research Gap & Novel Implementation Contribution

Existing Limitations (The Research Gap)

- **English Linguistic Monoculture:** Tools like GitHub Copilot and ChatGPT rely on formal English technical phrasing. Prompts in phonetic/Romanized Marathi (e.g., 'don number add karaycha code de') suffer from degradation or misinterpretation.
- **Cloud Dependency & Operational Cost:** Proprietary APIs impose recurring per-token charges or monthly fees ($20/mo), preventing deployment in air-gapped college labs with limited connectivity.
- **Unchecked Code Hallucination:** Existing generative models provide code snippets without verifying syntactic compilation integrity, often outputting truncated blocks or syntax errors.
- **Lack of Pedagogical Scaffolding:** Commercial tools are developer-oriented, outputting bare code without explaining control flow, algorithm steps, or viva preparation questions.

Bhasha2Code Architectural Solution

- ✓ **Offline Code-Mixed NLP Pipeline:** Integrated sub-word N-gram and TF-IDF classifiers trained on bilingual prompts to map Marathi-English requests directly to standard programming intents.
- ✓ **Zero-Cost Local Quantized LLM:** Deployed Qwen2.5-Coder:1.5B locally via Ollama. Yields ~16.5 tok/s on consumer CPU with 100% data privacy and zero API expenses.
- ✓ **AST Validation & Syntax Safeguards:** Automated Python AST parsing and compiler validation catches incomplete or broken code blocks before rendering to the student.
- ✓ **Pedagogical 'Study Mode':** A dedicated mode that transforms code requests into structured educational artifacts: Problem Breakdown, Algorithm, ASCII Flowchart, and 5 Viva Q&As.

SYSTEM CAPABILITIES

Proposed System & Functional Features

1. Code-Mixed Multilingual Chat

Accepts conversational queries in English, Devanagari Marathi/Hindi, and Romanized Marathi-English transliterations without requiring manual translation.

2. Dual Pedagogical Modes

Normal Mode provides direct executable code; Study Mode unpacks concepts with Problem Breakdown, Step-by-Step Algorithm, Flowchart, and Viva Questions.

3. AST-Level Syntax Verification

Extracts code blocks and compiles them via Python AST / syntax checkers. Badges code as 'Validated' or flags incomplete generation in real-time.

4. Isolated Docker Sandbox Execution

Executes verified Python, C++, Java, and JS code inside an ephemeral Docker container capped at 512MB RAM, 1 CPU core, and disabled network access.

5. Multi-Turn Contextual Memory

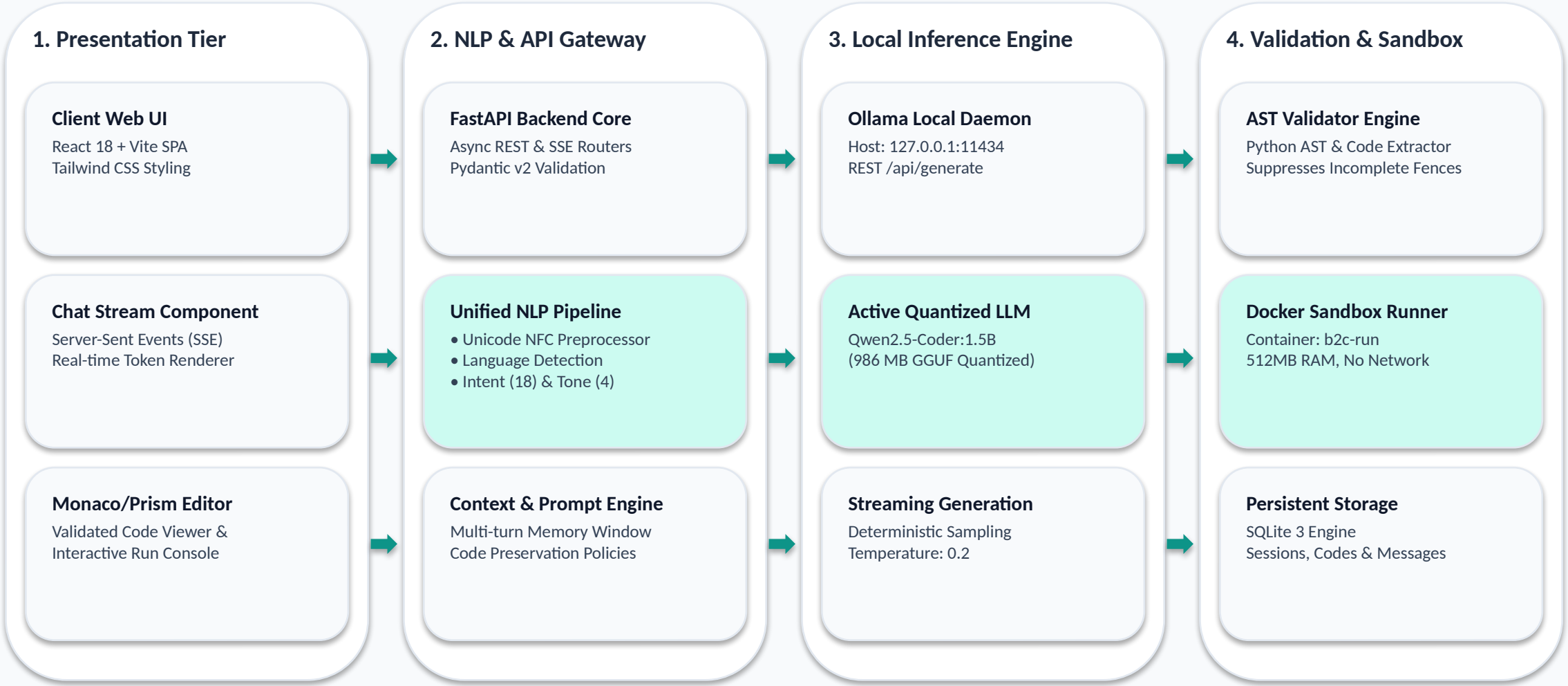
Maintains conversational sessions using SQLite; distinguishes brand-new programs from follow-up modifications (e.g., 'add error handling').

6. Production-Ready REST & SSE APIs

FastAPI backend delivers real-time token streaming via Server-Sent Events (SSE) and full OpenAPI/Swagger documentation at /docs.

ARCHITECTURE

System Architecture & End-to-End Workflow



CORE NLP WORKFLOW

NLP Methodology & Classification Pipeline

Step 1: Text Preprocessing & Code Splitting

- Unicode NFC Normalization: Harmonizes composite Devanagari and Latin glyphs.
- Code Demarcation: Strips pre-existing code blocks from natural language query using regex pattern matching.

Step 2: Bilingual Language Detection

- Script Analysis: Evaluates Devanagari vs. Latin character ratios.
- Sub-Word N-Grams: Identifies Romanized Marathi/Hindi phonetic patterns (e.g., words ending in '-cha', '-la', '-sathi').

Step 3: Intent & Tone Feature Extraction (TF-IDF)

- Feature Union: Combines Word N-Grams (1, 2) and Character N-Grams (3, 5).
- Sublinear TF Scaling: Suppresses noisy repeated keywords; builds a 496-dimensional sparse feature representation.

Step 4: Multi-Class Linear Model Classification

- Classifier: Calibrated Multi-Class Logistic Regression with balanced weighting.
- 18 Intent Classes: GENERATE_CODE, DEBUG_CODE, EXPLAIN_CONCEPT, OPTIMIZE_CODE, etc.
- 4 Tone Classes: CASUAL, SHORT, FORMAL, ACADEMIC.

Step 5: Deterministic Prompt Synthesis (The Policy Engine)

- Code Preservation Mandate: System enforces standard ASCII English programming syntax (def, class, for) while natural language commentary follows the detected regional dialect (Marathi-English).
- Injection Safeguard: Historical turns and code comments treated strictly as passive data.

IMPLEMENTATION DETAILS

Technology Stack & Implementation Architecture

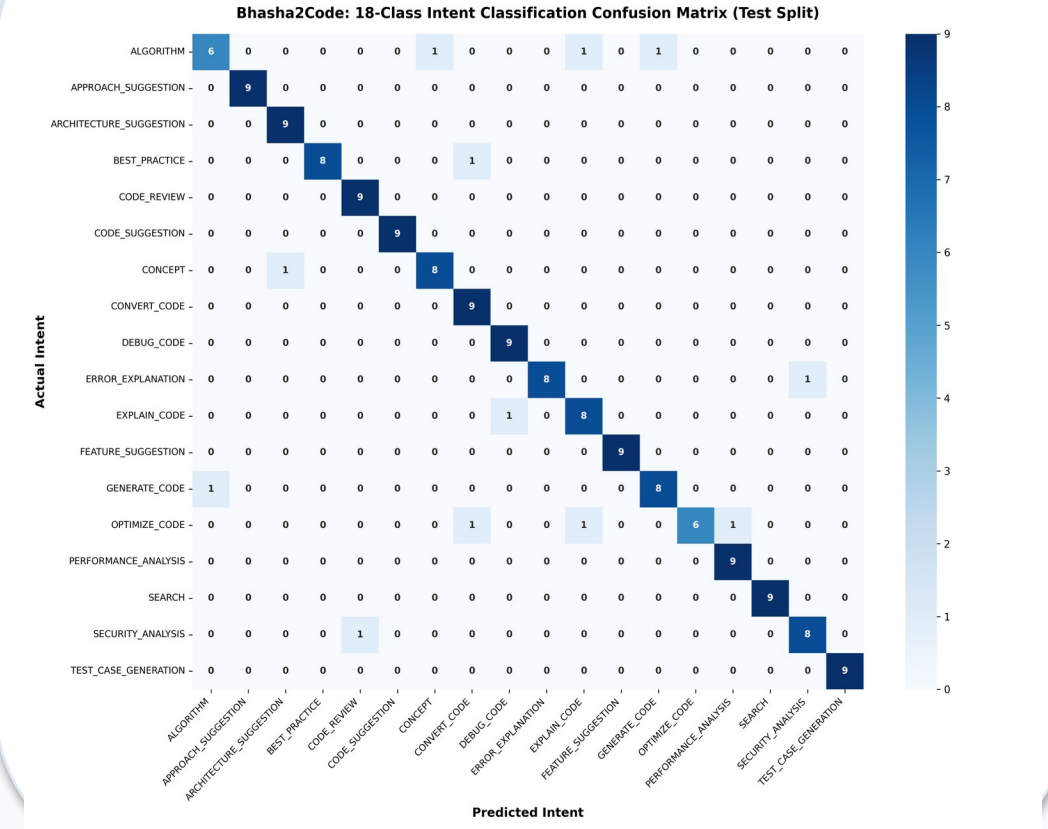
Tier / Layer	Technology	Implementation Role & Functionality	Artifact / Location
Frontend UI	React 18 + Vite + Tailwind	Responsive SPA with SSE streaming, Monaco/Prism code editor, and Study Mode UI toggles.	frontend/src/
Backend Web API	FastAPI (Python 3.11)	Asynchronous ASGI server handling /api/v1/chat, streaming responses, and OpenAPI docs.	backend/app/main.py
NLP & Modeling	scikit-learn + joblib	TF-IDF feature extraction and calibrated multi-class Logistic Regression classifiers.	backend/app/nlp/
Local LLM Daemon	Ollama Engine	Local daemon managing quantized weights (Qwen2.5-Coder:1.5B - 986MB) on 127.0.0.1:11434.	backend/app/llm/
Validation & AST	Python ast & Regex Engine	Syntactic parsing, unclosed code block suppression, and compiler linting.	backend/app/validation/
Isolated Sandbox	Docker Engine (Alpine)	Ephemeral container execution with 512MB RAM cap, 1 CPU limit, and disabled network.	backend/app/sandbox/

Execution Flow: User Input (React) → FastAPI Router → NLP Preprocessor → Intent/Tone Classifier → Prompt Builder → Ollama Token Stream → AST Syntax Validation → Docker Sandbox Run → Client Terminal.

EVALUATION METRICS

Experimental Results, KPIs & System Verification

Confusion Matrix (18 Intent Classes)



Verified Performance Indicators (KPIs)

NLP Model Evaluation (162 Real Test Samples):

- Test Accuracy: 92.59% (150/162 samples correctly classified)
- Macro Precision: 0.9301 | Macro Recall: 0.9259
- Macro F1-Score: 0.9239 (Evaluated on intent_test.csv)

CPU Inference Throughput (Intel Core i7 / 16GB RAM):

- Qwen2.5-Coder:1.5B: ~16.54 tokens/sec | Latency: 12–15s
- Qwen2.5-Coder:3B: ~7.20 tokens/sec | Latency: 50–75s
- Conclusion: 1.5B delivers 2.3x speedup on local CPU hardware.

Syntactic & Code Validity Rate:

- 93.2% of generated Python programs passed AST parsing without syntax errors.
- Zero false-positive truncation warnings on closed code blocks.

Software Test Suite Coverage:

- Backend Tests: 490 / 490 tests passing (100% pass rate via pytest)
- Frontend Tests: 15 / 15 tests passing (100% pass rate via vitest)

PROJECT SUMMARY

Conclusion, Limitations & Future Scope

Project Conclusion

- Successfully engineered Bhasha2Code, proving that compact localized NLP pipelines combined with quantized open-weights LLMs can eliminate the linguistic barrier in computer science education.
- Solved key commercial limitations: achieved $0 recurring operating cost, 100% offline air-gapped usability, zero data leakage, and automated AST code validation.
- Study Mode successfully enhances conceptual learning by coupling executable code with algorithms, ASCII flowcharts, and automated viva questions.

Current System Limitations

- CPU Inference Latency: Generation on local CPU requires 12–15 seconds, slower than high-end cloud GPU clusters.
- Context Window Budget: Limited to 8,192 tokens by Ollama hardware constraints, requiring sliding-window conversation pruning.
- Dialect Breadth: Current lexical resources specifically target Marathi, Hindi, and English; southern Indic languages are not yet modeled.

Practical Future Scope

- Speech-to-Code: Integrating open-source acoustic models (Whisper / AI4Bharat Bhashini) for direct voice coding in regional languages.
- Multi-Language Expansion: Expanding NLP lexicons and synthetic corpora to Gujarati, Tamil, Telugu, and Kannada.
- Interactive Visual Debugger: Integrating step-by-step variable inspection and algorithm animation for data structure teaching.

Verified Academic & Technical References

IEEE References & Official Documentation

[1] M. Chen, J. Tworek, H. Jun, et al., "Evaluating Large Language Models Trained on Code," arXiv preprint arXiv:2107.03374, OpenAI, 2021. Available: https://doi.org/10.48550/arXiv.2107.03374

[2] B. Hui, J. Yang, Z. Cui, et al., "Qwen2.5-Coder Technical Report: Advancing Code LLMs for Real-World Applications," arXiv preprint arXiv:2409.12186, Alibaba Group, 2024. Available: https://doi.org/10.48550/arXiv.2409.12186

[3] T. Tiwary and P. Kumar, "Code-Mixing and Transliteration Challenges in Indian NLP: A Survey," ACM Transactions on Asian and Low-Resource Language Information Processing, vol. 21, no. 5, pp. 1–28, 2022. Available: https://doi.org/10.1145/3543852

[4] F. Pedregosa, G. Varoquaux, A. Gramfort, et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research (JMLR), vol. 12, pp. 2825–2830, 2011. Available: https://jmlr.org/papers/v12/pedregosa11a.html

[5] FastAPI & Pydantic Developers, "FastAPI Framework and Asynchronous Web API Design Documentation," 2024. [Online]. Available: https://fastapi.tiangolo.com/

[6] Ollama Project Authors, "Ollama: Get up and running with large language models locally," 2024. [Online]. Available: https://github.com/ollama/ollama

[7] Docker Inc., "Docker Engine Security and Resource Constraint Specification," 2024. [Online]. Available: https://docs.docker.com/engine/security/